

Microservices Project

Continuous Assessment Project – Iteration 1

Advanced Microservices

Weighting: 30% of module mark

Submission Type: Individual Project

Submission: Code repository + short technical report

1. Overview

This continuous assessment project requires students to design and implement a microservice-based system demonstrating the use of distributed data management patterns in a realistic application scenario.

Students will choose their own project domain, but must implement a system of sufficient complexity to demonstrate understanding of:

- Microservice architecture
- API design
- Distributed data management patterns
- Failure handling in distributed systems

The project will be completed in two iterations across the semester.

This brief describes the requirements for Iteration 1.

2. Learning Objectives

This assessment is designed to evaluate your ability to:

- Design a microservice-based architecture
- Implement distributed data patterns
- Manage data consistency across services
- Design and test service APIs
- Reason about failure scenarios in distributed systems

3. Project Requirements – Iteration 1

Students must implement a multi-service application that includes the following architectural components.

3.1 Microservice Architecture

Your system must consist of multiple independently deployed services.

Minimum expectations:

- At least 3 microservices
- Each service must:
 - Have its own data store

- Expose functionality via an API
- Perform a distinct business responsibility

Examples of services might include:

- Order management
- Payment processing
- Inventory management
- Booking systems
- Account management
- Content publishing

The exact domain is your choice.

3.2 API Interface

Your system must expose a client-facing API that allows external clients to interact with the system. Your API should expose a number of business functions to clients.

Implement either:

- Implement an API using a framework with validation and asynchronous behaviour

or:

- A GraphQL API using Strawberry

Students are not required to build a client application, but must demonstrate functionality using tools such as:

- Postman
- Curl
- REST clients

API behaviour must be clearly demonstrated through test calls.

3.3 Distributed Data Patterns

Your implementation must demonstrate the following patterns.

Saga Pattern

Your system must implement a distributed workflow involving multiple services.

Requirements:

- Implement a multi-service transaction using a Saga
- Demonstrate how the workflow progresses between services
- Include failure handling using compensating actions

Example:

A booking system might:

1. Reserve a resource
2. Process payment
3. Confirm booking

If payment fails, the reservation must be released.

Transactional Outbox Pattern

Your system must demonstrate how events are reliably published after database updates.

Requirements:

- Implement an outbox table
- Write events to the outbox within the same database transaction as state updates
- Implement a mechanism that publishes events from the outbox to a message broker

This pattern should prevent dual-write inconsistencies.

CQRS (Command Query Responsibility Segregation)

Your system must separate:

- Write operations (commands)
- Read operations (queries)

Requirements:

- Implement distinct write and read models
- Queries should use optimised read projections
- Data must be synchronised through event-driven updates

Students should explain why this separation is beneficial in their system.

3.4 Messaging / Event Communication

Services must communicate using asynchronous messaging where appropriate.

Typical architecture:

```
1 | Service → Database + Outbox → Message Broker → Other Services
2 |
```

Acceptable technologies may include:

- Kafka
- Redis

Students must demonstrate:

- Event publication
- Event consumption
- Event-driven updates to read models

3.5 Data Ownership

Each microservice must own its own data.

Requirements:

- No shared databases
- Services communicate via APIs or events
- Data consistency managed using patterns introduced in the module

4. Project Complexity Expectations

Your project must be sufficiently complex to demonstrate your understanding of the module concepts.

An appropriate project should include:

- Multiple services interacting through events
- A distributed workflow involving several services
- At least one scenario where failure handling is required
- A meaningful read model created through CQRS

Simple CRUD applications with minimal interactions will not receive high marks.

5. Deliverables

Students must submit the following:

1. Source Code

A repository containing:

- All microservices
- API implementation
- Messaging components
- Configuration and setup instructions

The repository must include a clear README explaining:

- How to run the system
- How to test the APIs

2. Technical Report (Approx. 1500–2000 words)

The report should describe:

System Overview

- Application domain
- System functionality

Architecture

- Microservices included
- Service responsibilities
- Communication model

Distributed Data Patterns

Explain how your system implements:

- Saga
- Transactional Outbox
- CQRS

Failure Handling

Describe:

- Possible failure scenarios
- How your architecture handles them

API Description

Describe the main API endpoints and demonstrate example requests.

6. Demonstration of Functionality

Students must provide evidence that the system works, such as:

- API test examples (Postman screenshots or exported collections)
- Logs showing event processing
- Demonstration of successful and failed Saga execution

7. Use of AI

You may use AI to generate code, but you must declare, document and capture prompts used into the documentation. The prompts used should be at a level that they demonstrate understanding of the patterns being built. As always, a oral assessment may be conducted and marks will only be awarded for code you can explain - you must understand everything that is generated.

Do not use AI for generating your report. This is where you demonstrate your understanding and achievements against the learning outcomes. It must be in your own words.

8. Iteration 2 (Preview)

In the second iteration of this project, students will extend their system to include additional advanced microservice infrastructure, such as:

- Service mesh functionality
- Security mechanisms
- gRPC
- Other cross-cutting considerations

Further details will be provided in a later brief.